Web Engineering (CSC-324) Instructor: Syed Nouman Ali Shah

Assignment: Assignment 3 Max Marks: 50

Date: November 5, 2025, Submission Date: November 15, 2025

Assignment Theme

"Course Registration & Enrollment Mini-System"

Context: You are asked to build a small Course Registration & Enrollment backend and a simple React front-end using Spring Boot + Spring Data JPA + REST. This assignment extends the concepts of IoC/DI and now focuses on entities, repositories, business rules, and live integration with a React client.

You will model a Student–Course–Registration domain and expose it via REST APIs that your React app will consume.

Target Domain Model (Minimum Requirements)

Use the following minimal entities (you may add more if you like):

1. Student
   id (Long, PK)
   rollNo (String, unique)
   name (String, not null)
   email (String, unique, valid email)
   program (String, e.g., "BSCS", "BSE")
   semester (int)

2. Course
   id (Long, PK)
   code (String, unique, e.g., "CSC-324")
   title (String, not null)
   creditHours (int)
   maxSeats (int)
   prerequisite (self-reference, optional – another Course)

3. Registration
   id (Long, PK)
   student (Many-to-One → Student)

course (Many-to-One → Course)
semester (String, e.g. “Fall 2025”)
status (String: “REGISTERED”, “DROPPED”, “COMPLETED”)
grade (String, optional for now, e.g., “A”, “B+”)

Part 1 – Spring Data JPA Domain & Repositories (15 marks)

Task 1: Entity Design & Validation

Objective: Create proper entities and map relationships using JPA.

Instructions:

1. Create the three entities: Student, Course, Registration.
2. Use appropriate JPA annotations:
   - @Entity, @Id, @GeneratedValue
   - @ManyToOne, @OneToMany, @JoinColumn, @ManyToOne(optional = true) for prerequisites.
3. Add basic validation annotations where appropriate:
   - @NotBlank / @NotNull
   - @Email
   - @Min, @Max (for credit hours / semester)
4. Create a bidirectional relationship (optional but recommended) between:
   - Student ↔ Registration
   - Course ↔ Registration
5. Use Lombok (@Data, @NoArgsConstructor, @AllArgsConstructor) if allowed in your project.

✅ Learning Outcome: Students understand entity design, relationships, and bean validation in Spring Boot.

Task 2: JPA Repositories & Sample Data

Objective: Use Spring Data JPA repositories to interact with the DB.

Instructions:

1. Create repository interfaces:

- StudentRepository extends JpaRepository<Student, Long>
- CourseRepository extends JpaRepository<Course, Long>
- RegistrationRepository extends JpaRepository<Registration, Long>

2. Add some derived query methods, e.g.:
   - List findByStudentId(Long studentId);
   - List findByCourseId(Long courseId);
   - List findBySemester(String semester);

3. In a CommandLineRunner or DataLoader class:
   - Create at least 5 students.
   - Create at least 5 courses, where at least one course has a prerequisite.
   - Create a few sample registrations (3–5).

✅ Learning Outcome: Students can define repository interfaces and seed the database.

Part 2 – RESTful APIs for Course Registration (20 marks)

Task 3: Basic CRUD Endpoints

Objective: Expose REST endpoints for Student and Course management.

Instructions:

1. Create controllers:
   - StudentController (base path: /api/students)
   - CourseController (base path: /api/courses)
2. Implement basic CRUD:
   - GET /api/students – list all students
   - GET /api/students/{id} – get single student
   - POST /api/students – add new student
   - PUT /api/students/{id} – update student
   - DELETE /api/students/{id} – delete student (soft delete optional)
   - Same style for courses: /api/courses
3. Return appropriate HTTP status codes:
   - 201 CREATED for successful POST
   - 200 OK for GET/PUT
   - 204 NO CONTENT for DELETE
   - 404 NOT FOUND when a record is missing

✅ Learning Outcome: Students can design clean REST APIs with JPA and proper status codes.

Task 4: Registration Endpoints with Business Rules

Objective: Implement registration logic with at least three business rules.

Required Business Rules (minimum):

1. A student cannot register in more than 5 active courses (status REGISTERED) in the same semester.
2. A course cannot exceed its maxSeats for a given semester.
3. If a course has a prerequisite, the student must have a COMPLETED registration for that prerequisite course before registering.

Instructions:

1. Create RegistrationService and RegistrationController (base path: /api/registrations).
2. Endpoints:

    - POST /api/registrations

    Request body should include: studentId, courseId, semester.

        - Apply the business rules above.
        - If a rule is violated, return:
            - 400 BAD REQUEST with a JSON error message.
    - GET /api/registrations – list all registrations.
    - GET /api/students/{id}/registrations – list all registrations of a student.
    - GET /api/courses/{id}/registrations – list all registrations for a course.

3. Use a DTO (e.g., RegistrationRequestDTO) instead of exposing entity directly for POST.

✅ Learning Outcome: Implementing domain rules in service layer with Spring Data and REST.

Task 5: Central Error Handling (Mini-Exception Layer)

Objective: Improve API quality using centralized exception handling.

Instructions:

1. Create custom exceptions, e.g.:
   - StudentNotFoundException
   - CourseNotFoundException
   - BusinessRuleViolationException

2. Create @RestControllerAdvice class:
   - Map the above exceptions to structured JSON responses, e.g.:

```json
{
  "timestamp": "2025-11-15T10:00:00",
  "status": 400,
  "error": "Business rule violated",
  "message": "Student already has 5 active courses in this semester"
}
```

3. Ensure all controllers rely on services that may throw these exceptions.

✅ Learning Outcome: Handling errors cleanly and returning consistent JSON to the frontend.

Part 3 – React Frontend: Registration UI (10 marks)

Task 6: Course Listing & Registration Form

Objective: Build a small React UI that consumes your REST APIs.

Instructions:

1. Create a React app (using Vite or Create React App).
2. Create a CourseList component:
   - On mount, call GET /api/courses.
   - Display courses in a table or Bootstrap cards (code, title, credit hours, max seats).
   - Include a filter input to filter by course code or title (client-side filter is fine).
3. For each course row/card, show a “Register” button:
   - On click, open a small form/modal to:

    - Select Student (dropdown populated from GET /api/students)
    - Choose Semester (e.g., text input or dropdown)
- On submit, call POST /api/registrations.
- Handle both success and error:
    - On success → show a success alert and maybe clear form.
    - On business rule violation → show error message from backend.

✅ Learning Outcome: Students can integrate React, forms, and backend APIs.

Task 7: Student Dashboard View

Objective: Show registrations for a selected student.

Instructions:

1. Create `StudentDashboard` component:
    - Dropdown or input to select a student (by rollNo or name).
    - On selection, call `GET /api/students/{id}/registrations`.
2. Display list of registrations in a table:
    - Columns: Course Code, Course Title, Semester, Status, Grade.
3. Add a small **status filter** (e.g., show only `REGISTERED` or `COMPLETED`).

✅ **Learning Outcome:** Students learn to build simple dashboards consuming multiple endpoints.

Part 4 – Optional Bonus (Up to 5 Extra Marks)

Students can earn **bonus marks** by implementing any **one or more** of the following:

1. **Pagination & Sorting (Backend + Frontend)**
    - Use `Pageable` in your Spring Data repositories for `/api/courses` and `/api/students`.
    - Add simple Next/Previous page controls on frontend.
2. **Profiles & H2/MySQL Switch**
    - Use **Spring profiles** (`dev`, `prod`) to switch between:
        - `H2` in-memory DB
        - `MySQL` (or any RDBMS)
    - Show which profile is active at `/api/envinfo`.
3. **Basic Authentication/Security**
    - Add very simple Spring Security:
        - Only authenticated users can perform `POST /api/registrations`.
        - `GET` endpoints remain public.

Submission Deliverables

Deliverable	Description
Backend (Spring Boot Project)	Zipped `src` folder with entities, repositories, services, controllers, exception handlers, and sample data loader. Include `application.properties` / `application.yml`.
Frontend (React App)	Zipped React project (only `src` folder is also acceptable) showing Course listing, Register form, and Student dashboard.
Short Report (2–4 pages)	PDF/Word document describing: (a) Your domain model and relationships, (b) Business rules implemented and where, (c) Sample screenshots of successful registration and failure (business rule violation), (d) Any optional/bonus features you completed.

Marking Breakdown (Suggested) Part 1 – Entities & Repositories: 15 marks

Part 2 – REST APIs & Business Rules: 20 marks

Part 3 – React Frontend Integration: 10 marks

Code Quality & Report: 5 marks

Bonus (Optional): up to +5 marks